RESTful API веб-сервиса «Волонтёрский центр»

1. Общие сведения

Описываются три основных пользовательских сценария, показанных на изображении с экранами пользовательского интерфейса:

1. Вход в систему — авторизация пользователя и получение доступа к личному кабинету.
2. Прохождение теста — просмотр списка доступных тестов, получение вопросов, отправка ответов и просмотр результатов.
3. Изменение записи на собеседование — просмотр текущей записи, выбор новой даты и времени, сохранение изменений и получение уведомления об успешном переносе.

1.1. Общие параметры API

Параметр	Значение
Стиль API	REST
Базовый URL	https://api.volunteer-center.ru/api/v1
Формат данных	JSON
Кодировка	UTF-8
Протокол	HTTPS
Авторизация	Bearer-токен
Версия API	v1

Все запросы с телом передают заголовок:

```
Content-Type: application/json
Accept: application/json
```

Для запросов, требующих авторизации, дополнительно передаётся заголовок:

```
Authorization: Bearer <access_token>
```

В качестве идентификаторов объектов используется UUID.

1.2. Основные ресурсы API

Ресурс	Назначение
`/auth`	Авторизация пользователя
`/auth/logout`	Выход из системы
`/me`	Информация о текущем пользователе
`/tests`	Список доступных тестов
`/tests/{testId}`	Информация о конкретном тесте
`/tests/{testId}/questions`	Вопросы теста
`/test-attempts`	Создание попытки прохождения теста
`/test-attempts/{attemptId}/answers`	Отправка ответов
`/test-attempts/{attemptId}/submit`	Завершение теста
`/test-attempts/{attemptId}`	Получение результата
`/interview-slots`	Доступные даты и время собеседования
`/me/interview`	Текущая запись пользователя
`/me/interview/reschedule`	Перенос записи на собеседование

2. Сценарий № 1. Вход в систему

2.1. Метод POST `/auth/login` — вход в систему

Назначение: проверка учётных данных пользователя и выдача токена доступа.

Метод не требует предварительной авторизации.

2.1.1. Входные параметры запроса

Наименование	Тип	Обязательность	Ограничения	Описание
`login`	string	Да	От 3 до 254 символов, без пробелов по краям	Логин или адрес электронной почты пользователя
`password`	string	Да	От 8 до 128 символов	Пароль пользователя

Пример запроса:

```
POST /api/v1/auth/login
Content-Type: application/json
{
  "login": "petr.petrov@example.ru",
  "password": "qwerty123"
}
```

2.1.2. Выходные параметры успешного ответа

Код ответа: `200 OK`.

Наименование	Тип	Обязательность	Ограничения	Описание
`accessToken`	string	Да	Непустая строка	Токен доступа
`tokenType`	string	Да	Допустимое значение — Bearer	Тип токена
`expiresIn`	integer	Да	Положительное число секунд	Время действия токена
`user.id`	string (UUID)	Да	Корректный UUID	Идентификатор пользователя
`user.fullName`	string	Да	От 1 до 200 символов	Фамилия, имя и отчество либо отображаемое имя
`user.email`	string	Да	Корректный адрес электронной почты	Электронная почта
`user.role`	string	Да	Например, volunteer или admin	Роль пользователя

Пример успешного ответа:

```
{
  "accessToken": "eyJhbGciOiJIUzI1N",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "user":
{
    "id": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
    "fullName": "Петр Петров",
    "email": "petr.petrov@example.ru",
    "role": "volunteer"
  }
}
```

2.1.3. Выходные параметры ответов с ошибками

Наименование	Тип	Обязательность	Ограничения	Описание
error	object	Да	Объект	Объект, содержащий информацию об ошибке
error.code	string	Да	Непустая строка	Код ошибки
error.message	string	Да	Непустая строка	Текстовое описание ошибки
error.details	array of object	Нет	Массив объектов	Дополнительная информация об ошибках в отдельных полях входных данных
error.details[].field	string	Да, если элемент details присутствует	Непустая строка	Наименование поля, в котором обнаружена ошибка
error.details[].message	string	Да, если элемент details присутствует	Непустая строка	Описание ошибки для соответствующего поля

Примеры ответов с ошибками:

401 Unauthorized — неверный логин или пароль.

```
{
  "error":
{
    "code": "INVALID_CREDENTIALS",
    "message": "Неверный логин или пароль"
  }
}
```

422 Unprocessable Content — некорректные входные данные.

```
{
  "error":
{
    "code": "VALIDATION_ERROR",
    "message": "Ошибка проверки входных данных",
    "details":
[
      {
        "field": "login",
        "message": "Поле обязательно для заполнения"
      }
    ]
  }
```

```
}
```

429 Too Many Requests — превышено количество попыток входа.

```
{
  "error":
{
    "code": "TOO_MANY_REQUESTS",
    "message": "Слишком много попыток входа. Повторите попытку позже"
  }
}
```

2.1.4. Алгоритм обработки запроса

1. Сервер принимает POST-запрос и проверяет наличие тела запроса в формате JSON.
2. Проверяет наличие обязательных полей login и password, их типы, длину и допустимость значений.
3. Проверяет ограничение частоты запросов для данного IP-адреса и учётной записи. При превышении лимита возвращает 429.
4. Ищет пользователя по логину или адресу электронной почты.
5. Проверяет, что учётная запись существует, активна и не заблокирована.
6. Сравнивает переданный пароль с хешем пароля, хранящимся в базе данных. Пароль в открытом виде не сохраняется.
7. Если проверка не пройдена, возвращает 401 с единым сообщением об ошибке, не раскрывая, существует ли указанный пользователь.
8. Если проверка пройдена, формирует подписанный токен доступа с идентификатором пользователя и сроком действия.
9. Формирует JSON-ответ с токеном и общедоступными данными пользователя.
10. Возвращает 200 OK.

2.2. Метод GET `/me` — получение информации о текущем пользователе

Назначение: загрузка профиля пользователя после входа в систему и отображение персонализированной информации в личном кабинете.

2.2.1. Входные параметры

Параметры в URL и теле запроса отсутствуют. Обязателен заголовок Authorization.

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Формат Bearer {token}	Токен доступа

2.2.2. Выходные параметры

Код ответа: `200 OK`.

Наименование	Тип	Обязательность	Ограничения	Описание

`id`	string (UUID)	Да	Корректный UUID	Идентификатор пользователя
`fullName`	string	Да	До 200 символов	Отображаемое имя
`email`	string	Да	Корректный адрес электронной почты	Электронная почта
`role`	string	Да	Значение из перечня ролей	Роль пользователя
`status`	string	Да	Например, active, blocked	Статус учётной записи

Пример запроса:

```
GET /api/v1/me
Authorization: Bearer eyJhbGciOiJIUzI1
```

Пример успешного ответа:

```
{
  "id": "a1b2c3d4-e5f6-4789-8123-456789abc",
  "fullName": "Петр Петров",
  "email": "petr.petrov@example.ru",
  "role": "volunteer",
  "status": "active"
}
```

Пример ответа с ошибкой:

```
{
  "error": {
    "code": "TOKEN_EXPIRED",
    "message": "Срок действия токена истёк"
  }
}
```

2.2.3. Алгоритм обработки запроса

1. Сервер получает токен из заголовка Authorization.
2. Проверяет наличие токена, его подпись, срок действия и допустимого издателя.
3. Извлекает идентификатор пользователя из токена.
4. Находит пользователя в базе данных.
5. Проверяет, что учётная запись существует и не заблокирована.
6. Получает разрешённые для пользователя данные профиля.

7. Исключает из ответа пароль, хеш пароля, служебные поля и другие конфиденциальные данные.
8. Возвращает 200 OK и JSON с информацией о пользователе. При недействительном токене возвращает 401, а при заблокированной учётной записи — 403.

2.3. Метод POST `/auth/logout` — выход из системы

Назначение: завершение сеанса пользователя.

2.3.1. Входные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Действующий токен Bearer	Токен текущего пользователя

Тело запроса отсутствует.

Пример запроса:

```
POST /api/v1/auth/logout
Authorization: Bearer eyJhbGciOiJIUzI1
```

Пример успешного ответа (`204 No Content`): тело ответа отсутствует.

Пример ответа с ошибкой:

```
{
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Требуется авторизация"
  }
}
```

2.3.2. Алгоритм обработки запроса

1. Сервер проверяет заголовок авторизации и валидность токена.
2. Сервер определяет текущего пользователя и идентификатор сеанса или токена.
3. Возвращает 204 No Content.

3. Сценарий № 2. Прохождение теста

Пользователь переходит на страницу тестов, просматривает доступные задания, открывает тест, отвечает на вопросы и отправляет ответы. После этого система сохраняет результат и отображает состояние прохождения.

Для этого сценария проектируются следующие методы:

Метод и URL	Назначение
`GET /tests`	Получение списка тестов

`GET /tests/{testId}`	Получение информации о тесте
`GET /tests/{testId}/questions`	Получение вопросов теста
`POST /test-attempts`	Начало прохождения теста
`PUT /test-attempts/{attemptId}/answers`	Сохранение ответов
`POST /test-attempts/{attemptId}/submit`	Завершение теста
`GET /test-attempts/{attemptId}`	Получение результата попытки

Все перечисленные методы требуют авторизации пользователя.

3.1. Метод GET `/tests` — получение списка тестов

Назначение: отображение доступных пользователю тестов на экране «Тесты».

3.1.1. Входные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Действующий Bearer-токен	Авторизация
`status`	string	Нет	available, in_progress, completed	Фильтрация по состоянию теста
`page`	integer	Нет	От 1, по умолчанию 1	Номер страницы
`limit`	integer	Нет	От 1 до 100, по умолчанию 20	Количество записей на странице

3.1.2. Выходные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`items`	array	Да	Массив объектов тестов	Список тестов
`items[].id`	string (UUID)	Да	UUID	Идентификатор теста
`items[].title`	string	Да	От 1 до 200 символов	Название теста
`items[].description`	string	Нет	До 2000 символов	Описание теста

items[].status	string	Да	available, in_progress, completed	Состояние теста для пользователя
items[].durationMinutes	integer	Да	Положительное число	Продолжительность теста в минутах
items[].questionCount	integer	Да	Неотрицательное число	Количество вопросов
page	integer	Да	От 1	Текущая страница
limit	integer	Да	От 1 до 100	Размер страницы
total	integer	Да	Неотрицательное число	Общее количество результатов

Пример запроса:

```
GET /api/v1/tests?status=available&page=1&limit=20
Authorization: Bearer <access_token>
```

Пример успешного ответа (200 OK):

```
{
  "items": [
    {
      "id": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
      "title": "Основы волонтёрской деятельности",
      "description": "Проверка знаний правил работы волонтёра",
      "status": "available",
      "durationMinutes": 30,
      "questionCount": 15
    },
    {
      "id": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
      "title": "Безопасность при проведении мероприятий",
      "description": "Проверка знаний требований безопасности",
      "status": "available",
      "durationMinutes": 20,
      "questionCount": 10
    }
  ],
  "page": 1,
  "limit": 20,
  "total": 2
}
```

Пример ошибки:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Некорректный параметр запроса",
    "details": [
      {
        "field": "page",
        "message": "Значение должно быть не меньше 1"
      }
    ]
  }
}
```

3.1.3. Алгоритм обработки запроса

1. Проверить наличие и валидность токена пользователя.
2. Проверить типы и допустимые значения параметров status, page и limit.
3. Определить пользователя, от имени которого выполняется запрос.
4. Получить список опубликованных тестов, доступных данному пользователю с учётом его роли и условий допуска.
5. Если указан фильтр status, определить состояние каждого теста относительно текущего пользователя и применить фильтр.
6. Выполнить пагинацию, ограничив количество возвращаемых записей значением limit.
7. Сформировать для каждого теста краткую информацию без правильных ответов.
8. Вернуть список тестов и общее количество записей с кодом 200 OK.
9. При некорректных параметрах вернуть 422 Unprocessable Content, при отсутствии авторизации — 401 Unauthorized.

3.2. Метод GET `/tests/{testId}` — получение информации о тесте

Назначение: отображение описания выбранного теста перед началом прохождения.

3.2.1. Входные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Действующий Bearer-токен	Авторизация
`testId`	string (UUID)	Да	Корректный UUID в пути запроса	Идентификатор теста

3.2.2. Выходные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`id`	string	Да	UUID	Идентификатор теста

	(UUID)			
`title`	string	Да	До 200 символов	Название
`description`	string	Нет	До 2000 символов	Описание
`durationMinutes`	integer	Да	Больше 0	Лимит времени прохождения
`questionCount`	integer	Да	Неотрицательное число	Количество вопросов
`maxAttempts`	integer	Да	Больше 0	Максимальное количество попыток
`passingScore`	number	Да	От 0 до 100	Проходной процент
`status`	string	Да	Например, available, completed	Состояние теста для пользователя

Пример запроса:

```
GET /api/v1/tests/a1b2c3d4-e5f6-4789-8123-456789abcd0
Authorization: Bearer <access_token>
```

Пример успешного ответа:

```
{
  "id": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
  "title": "Основы волонтёрской деятельности",
  "description": "Проверка знаний правил работы волонтёра",
  "durationMinutes": 30,
  "questionCount": 15,
  "maxAttempts": 3,
  "passingScore": 70,
  "status": "available"
}
```

Пример ошибки (`404 Not Found`):

```
{
  "error": {
    "code": "TEST_NOT_FOUND",
    "message": "Тест не найден"
  }
}
```

3.2.3. Алгоритм обработки запроса

1. Проверить авторизацию пользователя.

2. Проверить корректность формата testId.
3. Найти тест в базе данных.
4. Проверить, опубликован ли тест и доступен ли он пользователю.
5. Получить параметры теста и состояние пользователя: количество попыток, доступность прохождения и текущий статус.
6. Не возвращать правильные ответы и другие внутренние данные.
7. Вернуть тест с кодом 200 OK.
8. Если тест отсутствует или недоступен, вернуть 404 Not Found. Если тест существует, но прохождение запрещено политикой доступа, вернуть 403 Forbidden.

3.3. Метод GET `/tests/{testId}/questions` — получение вопросов

Назначение: загрузка вопросов и вариантов ответов для прохождения выбранного теста.

3.3.1. Входные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Действующий Bearer-токен	Авторизация
`testId`	string (UUID)	Да	UUID	Идентификатор теста

3.3.2. Выходные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`testId`	string (UUID)	Да	UUID	Идентификатор теста
`questions`	array	Да	Массив объектов вопросов	Список вопросов
`questions[].id`	string (UUID)	Да	UUID	Идентификатор вопроса
`questions[].text`	string	Да	От 1 до 5000 символов	Текст вопроса
`questions[].type`	string	Да	single_choice, multiple_choice, text	Тип ответа
`questions[].options`	array	Да	Может быть пустым для текстового вопроса	Варианты ответа
`questions[].options[].id`	string (UUID)	Да, для каждого варианта	UUID	Идентификатор варианта

questions[].options[].text	string	Да, для каждого варианта	До 1000 символов	Текст варианта

Пример запроса:

```
GET /api/v1/tests/a1b2c3d4-e5f6-4789-8123-456789abcd0/questions
Authorization: Bearer <access_token>
```

Пример успешного ответа:

```
{
  "testId": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
  "questions": [
    {
      "id": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
      "text": "Как следует поступить при возникновении угрозы
безопасности?",
      "type": "single_choice",
      "options": [
        {
          "id": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
          "text": "Сообщить ответственному лицу"
        },
        {
          "id": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
          "text": "Игнорировать ситуацию"
        }
      ]
    }
  ]
}
```

Пример ошибки (403 Forbidden):

```
{
  "error": {
    "code": "TEST_ACCESS_DENIED",
    "message": "У пользователя нет доступа к вопросам данного теста"
  }
}
```

3.3.3. Алгоритм обработки запроса

1. Проверить токен доступа и статус пользователя.
2. Проверить формат идентификатора теста.
3. Найти тест и проверить, разрешено ли пользователю просматривать его вопросы.
4. Проверить статус теста: например, тест должен быть опубликован, а пользователь должен иметь право начать его прохождение.
5. Получить список вопросов и связанных с ними вариантов ответов.

6. Сформировать ответ, сохраняя порядок вопросов, заданный настройками теста.
7. Вернуть 200 OK. Если тест не найден, вернуть 404; если доступ запрещён, вернуть 403.

3.4. Метод POST `/test-attempts` — начало прохождения теста

Назначение: создание новой попытки прохождения теста.

3.4.1. Входные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Действующий Bearer-токен	Авторизация
`testId`	string (UUID)	Да	UUID	Идентификатор теста

Тело запроса:

```
{
  "testId": "a1b2c3d4-e5f6-4789-8123-456789abcd0"
}
```

3.4.2. Выходные параметры

Код ответа: `201 Created.`

Наименование	Тип	Обязательность	Ограничения	Описание
`id`	string (UUID)	Да	UUID	Идентификатор попытки
`testId`	string (UUID)	Да	UUID	Идентификатор теста
`status`	string	Да	in_progress	Состояние попытки
`startedAt`	string (date-time)	Да	ISO 8601	Время начала
`expiresAt`	string (date-time)	Да	Позже startedAt	Предельное время завершения

Пример успешного ответа:

```
{
  "id": "6",
  "testId": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
  "status": "in_progress",
  "startedAt": "2026-10-21T11:00:00+03:00",
```

```
  "expiresAt": "2026-10-21T11:30:00+03:00"
}
```

Пример ошибки (`409 Conflict`):

```
{
  "error": {
    "code": "ATTEMPTS_LIMIT_REACHED",
    "message": "Достигнуто максимальное количество попыток прохождения
теста"
  }
}
```

3.4.3. Алгоритм обработки запроса

1. Проверить авторизацию пользователя.
2. Проверить наличие testId и корректность его формата.
3. Найти тест, проверить его публикацию, доступность и требования к допуску.
4. Проверить количество предыдущих попыток пользователя.
5. Проверить, нет ли уже активной попытки, если правила теста запрещают одновременное прохождение.
6. При превышении лимита попыток вернуть 409 Conflict.
7. Создать запись попытки в базе данных со статусом in_progress.
8. Зафиксировать время начала и рассчитать expiresAt на основании продолжительности теста.
9. Вернуть 201 Created с данными новой попытки и заголовком Location, указывающим на созданный ресурс.
10. Если параллельно поступают несколько запросов на начало теста, ограничения должны проверяться транзакционно, чтобы пользователь не получил лишние попытки.

3.5. Метод PUT `/test-attempts/{attemptId}/answers` — сохранение ответов

Назначение: сохранение ответов пользователя во время прохождения теста.

Метод PUT используется здесь для полной замены текущего набора ответов в рамках попытки. Если необходимо отправлять только отдельные ответы, альтернативой является PATCH или PUT /test-attempts/{attemptId}/answers/{questionId}.

3.5.1. Входные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Действующий Bearer-токен	Авторизация
`attemptId`	string (UUID)	Да	UUID	Идентификатор

				попытки
`answers`	array	Да	Не более n вопросов в тесте	Массив ответов
`answers[].questionId`	string (UUID)	Да	UUID вопроса из текущего теста	Идентификатор вопроса
`answers[].selectedOptionIds`	array of UUID	Для вопросов с вариантами ответа	Для single_choice — не более одного элемента	Выбранные варианты
`answers[].text`	string	Для текстовых вопросов	До 5000 символов	Текстовый ответ

Пример запроса:

```
PUT /api/v1/test-attempts/66666666-6666-4666-8666-666666666666/answers
Authorization: Bearer <access_token>
Content-Type: application/json
```

```
{
  "answers": [
    {
      "questionId": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
      "selectedOptionIds": [
        "a1b2c3d4-e5f6-4789-8123-456789abcd0"
      ]
    }
  ]
}
```

3.5.2. Выходные параметры

Код ответа: `200 OK`.

Наименование	Тип	Обязательность	Ограничения	Описание
`attemptId`	string (UUID)	Да	UUID	Идентификатор попытки
`status`	string	Да	in_progress	Статус попытки
`savedCount`	integer	Да	Неотрицательное число	Количество сохранённых ответов
`updatedAt`	string (date-time)	Да	ISO 8601	Время последнего сохранения

Пример успешного ответа:

```
{
  "attemptId": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
  "status": "in_progress",
  "savedCount": 1,
  "updatedAt": "2026-10-21T11:10:00+03:00"
}
```

Пример ошибки (`422 Unprocessable Content`):

```
{
  "error": {
    "code": "INVALID_ANSWER",
    "message": "Ответ не соответствует типу вопроса",
    "details": [
      {
        "field": "answers[0].selectedOptionIds",
        "message": "Для вопроса с одним вариантом ответа допускается
только один выбранный вариант"
      }
    ]
  }
}
```

3.5.3. Алгоритм обработки запроса

1. Проверить токен доступа.
2. Найти попытку по attemptId и убедиться, что она принадлежит текущему пользователю.
3. Проверить статус попытки. Ответы разрешается сохранять только в состоянии in_progress.
4. Проверить время окончания попытки. Если время истекло, запретить дальнейшее изменение ответов и применить предусмотренное правилами теста завершение.
5. Проверить структуру массива answers, наличие обязательных полей и соответствие типов данных.
6. Для каждого ответа проверить, что вопрос относится к указанному тесту.
7. Для вопросов с вариантами проверить принадлежность каждого выбранного варианта вопросу, количество выбранных вариантов и допустимость пустого ответа.
8. Для текстовых вопросов проверить длину и допустимость содержимого.
9. Сохранить набор ответов в транзакции. При использовании полной замены удалить или обновить ранее сохранённые ответы в соответствии с переданным массивом.
10. Обновить время сохранения попытки.
11. Вернуть 200 OK с количеством сохранённых ответов. Если один из ответов не прошёл проверку, вернуть ошибку и не сохранять частично обработанный набор.

3.6. Метод POST `/test-attempts/{attemptId}/submit` — завершение теста

Назначение: окончательная отправка ответов, оценивание и фиксация результата прохождения теста.

3.6.1. Входные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Действующий Bearer-токен	Авторизация
`attemptId`	string (UUID)	Да	UUID	Идентификатор попытки

Тело запроса отсутствует: сервер использует ответы, предварительно сохранённые методом `PUT /test-attempts/{attemptId}/answers`.

Пример запроса:

```
POST /api/v1/test-attempts/a1b2c3d4-e5f6-4789-8123-456789abcd0/submit
Authorization: Bearer <access_token>
```

3.6.2. Выходные параметры

Код ответа: `200 OK`.

Наименование	Тип	Обязательность	Ограничения	Описание
`attemptId`	string (UUID)	Да	UUID	Идентификатор попытки
`status`	string	Да	`completed`	Статус попытки
`score`	number	Да	От 0 до 100	Итоговый процент правильных ответов
`correctAnswers`	integer	Да	От 0 до общего количества вопросов	Количество правильных ответов
`totalQuestions`	integer	Да	Неотрицательное число	Общее количество вопросов
`passed`	boolean	Да	`true` или `false`	Пройден ли тест
`completedAt`	string (date-time)	Да	ISO 8601	Время завершения

Пример успешного ответа:

```
{
  "attemptId": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
  "status": "completed",
  "score": 80,
  "correctAnswers": 12,
  "totalQuestions": 15,
  "passed": true,
  "completedAt": "2026-10-21T11:20:00+03:00"
}
```

Пример ошибки (`409 Conflict`):

```
{
  "error": {
    "code": "ATTEMPT_ALREADY_COMPLETED",
    "message": "Попытка прохождения теста уже завершена"
  }
}
```

3.6.3. Алгоритм обработки запроса

1. Проверить авторизацию и принадлежность попытки текущему пользователю.
2. Проверить, что попытка существует и имеет статус in_progress.
3. Если попытка уже завершена, не выполнять повторное оценивание с изменением результата. Вернуть ранее сохранённый результат либо 409 Conflict, согласно принятому контракту API.
4. Проверить время прохождения и наличие ответов. Если правила допускают завершение с пропусками, считать пропущенные вопросы неотвеченными; в противном случае вернуть ошибку о незаполненных вопросах.
5. Получить ответы из базы данных и сопоставить их с правильными ответами, хранящимися на сервере.
6. Рассчитать число правильных ответов и итоговый процент по установленным правилам оценивания.
7. Сравнить результат с проходным порогом теста.
8. В рамках транзакции сохранить итоговые баллы, статус completed, признак прохождения и время завершения.
9. При необходимости обновить состояние теста для пользователя и сформировать событие для уведомления или отображения результата в личном кабинете.
10. Вернуть 200 OK с итоговым результатом.

Для защиты от повторной отправки и сетевых повторов завершение попытки должно быть идемпотентным: повторный запрос не должен создавать вторую попытку или менять уже зафиксированный результат.

3.7. Метод GET `/test-attempts/{attemptId}` — получение результата теста

Назначение: отображение результата прохождения теста при повторном открытии страницы.

3.7.1. Входные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Действующий Bearer-токен	Авторизация
`attemptId`	string (UUID)	Да	UUID	Идентификатор попытки

3.7.2. Выходные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`id`	string (UUID)	Да	UUID	Идентификатор попытки
`testId`	string (UUID)	Да	UUID	Идентификатор теста
`status`	string	Да	in_progress, completed, expired	Состояние попытки
`score`	number или null	Да	От 0 до 100	Итоговый балл, если тест завершён
`passed`	boolean или null	Да	true или false	Результат прохождения
`startedAt`	string (date-time)	Да	ISO 8601	Время начала
`completedAt`	string (date-time) или null	Да	ISO 8601	Время завершения, если оно есть

Пример запроса:

```
GET /api/v1/test-attempts/a1b2c3d4-e5f6-4789-8123-456789abcd0
Authorization: Bearer <access_token>
```

Пример успешного ответа:

```
{
  "id": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
  "testId": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
  "status": "completed",
  "score": 80,
```

```
  "passed": true,
  "startedAt": "2026-10-21T11:00:00+03:00",
  "completedAt": "2026-10-21T11:20:00+03:00"
}
```

Пример ошибки (`404 Not Found`):

```
{
  "error": {
    "code": "ATTEMPT_NOT_FOUND",
    "message": "Попытка прохождения теста не найдена"
  }
}
```

3.7.3. Алгоритм обработки запроса

1. Проверить авторизацию пользователя.
2. Проверить корректность attemptId.
3. Найти попытку в базе данных.
4. Убедиться, что попытка принадлежит текущему пользователю либо что у пользователя есть административные права на просмотр.
5. Получить сохранённый статус, результат и временные метки.
6. Если попытка ещё выполняется, вернуть in_progress и не показывать нерассчитанный результат.
7. Если попытка завершена, вернуть сохранённые баллы и признак прохождения.
8. Если попытка не найдена или доступ к ней запрещён, вернуть соответствующий код ошибки.

4. Сценарий № 3. Изменение записи на собеседование

По схеме пользователь открывает запись на собеседование, переходит к выбору новой даты и времени, выбирает доступный слот и подтверждает перенос. Затем система показывает уведомление об успешном изменении записи.

Для данного сценария предусмотрены три метода:

Метод и URL	Назначение
`GET /me/interview`	Получение текущей записи
`GET /interview-slots`	Получение доступных слотов
`PUT /me/interview/reschedule`	Изменение даты и времени записи

Все методы требуют авторизации.

4.1. Метод GET `/me/interview` — получение текущей записи

Назначение: загрузка сведений о запланированном собеседовании пользователя.

4.1.1. Входные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Действующий Bearer-токен	Авторизация

Параметры URL и тело запроса отсутствуют.

4.1.2. Выходные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`id`	string (UUID)	Да	UUID	Идентификатор записи
`status`	string	Да	scheduled, completed, cancelled	Статус записи
`scheduledAt`	string (date-time)	Да	ISO 8601 с часовым поясом	Дата и время собеседования
`location`	string или null	Да	До 500 символов	Место проведения или ссылка на онлайн-собеседование
`canReschedule`	boolean	Да	true или false	Разрешён ли перенос
`updatedAt`	string (date-time)	Да	ISO 8601	Время последнего изменения

Пример запроса:

```
GET /api/v1/me/interview
Authorization: Bearer <access_token>
```

Пример успешного ответа:

```
{
  "id": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
  "status": "scheduled",
  "scheduledAt": "2026-10-25T14:00:00+03:00",
  "location": "Волонтёрский центр, кабинет 12",
  "canReschedule": true,
  "updatedAt": "2026-10-20T15:30:00+03:00"
}
```

Пример ошибки (`404 Not Found`):

```
{
  "error": {
    "code": "INTERVIEW_NOT_FOUND",
    "message": "У пользователя нет записи на собеседование"
  }
}
```

4.1.3. Алгоритм обработки запроса

1. Проверить токен доступа и определить идентификатор пользователя.
2. Найти активную запись на собеседование, связанную с этим пользователем.
3. Если запись отсутствует, вернуть 404 Not Found.
4. Получить дату и время записи, место проведения и текущий статус.
5. Проверить правила переноса: статус записи, время до начала собеседования, ограничения организатора и другие условия.
6. Сформировать значение canReschedule на основании действующих правил.
7. Вернуть данные записи с кодом 200 OK.

4.2. Метод GET `/interview-slots` — получение доступных слотов

Назначение: отображение доступных дат и времени в календаре выбора новой записи.

4.2.1. Входные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Действующий Bearer-токен	Авторизация
`from`	string (date-time)	Да	Корректная дата ISO 8601	Начало периода поиска
`to`	string (date-time)	Да	После from	Конец периода поиска
`timezone`	string	Нет	Допустимое название часового пояса IANA	Часовой пояс отображения, например Europe/Moscow

Для ограничения нагрузки максимальный период поиска установить в 30 дней.

4.2.2. Выходные параметры

Наименование	Тип	Обязательность	Ограничения	Описание

`items`	array	Да	Массив слотов	Доступные временные интервалы
`items[].id`	string (UUID)	Да	UUID	Идентификатор слота
`items[].startAt`	string (date-time)	Да	ISO 8601	Начало интервала
`items[].endAt`	string (date-time)	Да	Позже startAt	Конец интервала
`items[].location`	string	Да	До 500 символов	Место проведения или формат встречи
`items[].available`	boolean	Да	true или false	Доступность слота

Пример запроса:

```
GET /api/v1/interview-slots?from=2026-10-22T00:00:00%2B03:00&to=2026-11-05T23:59:59%2B03:00&timezone=Europe%2FMoscow
Authorization: Bearer <access_token>
```

Пример успешного ответа:

```
{
  "items": [
    {
      "id": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
      "startAt": "2026-10-26T10:00:00+03:00",
      "endAt": "2026-10-26T10:30:00+03:00",
      "location": "Волонтёрский центр, кабинет 12",
      "available": true
    },
    {
      "id": "99999999-9999-4999-8999-999999999999",
      "startAt": "2026-10-27T15:00:00+03:00",
      "endAt": "2026-10-27T15:30:00+03:00",
      "location": "Волонтёрский центр, кабинет 12",
      "available": true
    }
  ]
}
```

Пример ошибки (`422 Unprocessable Content`):

```
{
  "error": {
    "code": "INVALID_DATE_RANGE",
    "message": "Дата окончания должна быть позже даты начала, а период
```

```
не должен превышать 30 дней"
  }
}
```

4.2.3. Алгоритм обработки запроса

1. Проверить авторизацию пользователя.
2. Проверить наличие параметров from и to, корректность формата даты и соответствие часового пояса допустимым значениям.
3. Проверить, что to позже from, а запрашиваемый период не превышает установленный предел.
4. Получить опубликованные слоты собеседований, попадающие в указанный период.
5. Исключить прошедшие слоты, слоты вне расписания и интервалы, которые не могут быть назначены пользователю.
6. Проверить занятость каждого слота по данным системы бронирования.
7. Вернуть список доступных слотов и их идентификаторов с кодом 200 OK.
8. При некорректном периоде вернуть 422 Unprocessable Content.

4.3. Метод PUT `/me/interview/reschedule` — перенос собеседования

Назначение: изменение даты и времени существующей записи на собеседование на выбранный пользователем доступный слот.

Метод PUT используется для изменения состояния существующей записи. Повторная отправка одного и того же запроса не должна создавать новые записи или дублировать бронирование.

4.3.1. Входные параметры

Наименование	Тип	Обязательность	Ограничения	Описание
`Authorization`	string	Да	Действующий Bearer-токен	Авторизация пользователя
`slotId`	string (UUID)	Да	Корректный UUID	Идентификатор выбранного слота
`comment`	string	Нет	До 1000 символов	Комментарий пользователя о переносе
`Idempotency-Key`	string	Нет, рекомендуется	От 16 до 128 символов	Ключ для защиты от повторного выполнения запроса

Пример запроса:

```
PUT /api/v1/me/interview/reschedule
Authorization: Bearer <access_token>
```

```
Content-Type: application/json
Idempotency-Key: reschedule-20261021-001
```

```
{
  "slotId": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
  "comment": "Не смогу присутствовать в первоначальное время"
}
```

4.3.2. Выходные параметры

Код ответа: `200 OK`.

Наименование	Тип	Обязательность	Ограничения	Описание
`id`	string (UUID)	Да	UUID	Идентификатор записи
`status`	string	Да	scheduled	Статус записи после переноса
`scheduledAt`	string (date-time)	Да	ISO 8601	Новые дата и время собеседования
`location`	string или null	Да	До 500 символов	Место проведения
`updatedAt`	string (date-time)	Да	ISO 8601	Время изменения записи

Пример успешного ответа:

```
{
  "id": "a1b2c3d4-e5f6-4789-8123-456789abcd0",
  "status": "scheduled",
  "scheduledAt": "2026-10-26T10:00:00+03:00",
  "location": "Волонтёрский центр, кабинет 12",
  "updatedAt": "2026-10-21T11:25:00+03:00"
}
```

4.3.3. Примеры ошибок

404 Not Found — выбранный слот не найден.

```
{
  "error": {
    "code": "SLOT_NOT_FOUND",
    "message": "Выбранный слот не найден"
  }
```

```
}
```

409 Conflict — слот уже занят.

```
{
  "error": {
    "code": "SLOT_ALREADY_BOOKED",
    "message": "Выбранное время уже занято. Выберите другой слот"
  }
}
```

409 Conflict — перенос запрещён для текущего состояния записи.

```
{
  "error": {
    "code": "RESCHEDULE_NOT_ALLOWED",
    "message": "Изменение записи в текущем состоянии невозможно"
  }
}
```

4.3.4. Алгоритм обработки запроса

1. Авторизация. Проверить токен доступа и определить пользователя, от имени которого выполняется запрос.
2. Валидация. Проверить формат slotId, длину комментария и корректность ключа идемпотентности, если он передан.
3. Поиск записи. Получить текущую запись на собеседование пользователя. Если запись отсутствует, вернуть 404 Not Found.
4. Проверка возможности переноса. Убедиться, что запись находится в статусе scheduled, не завершена и не отменена, а также удовлетворяет правилам переноса.
5. Проверка слота. Найти слот по slotId, проверить его существование, принадлежность допустимому расписанию и отсутствие прошедшего времени.
6. Проверка занятости. Повторно проверить доступность слота непосредственно перед бронированием. Если он занят, вернуть 409 Conflict.
7. Транзакционное изменение. В рамках одной транзакции заблокировать выбранный слот, закрепить его за пользователем, изменить дату и время существующей записи и освободить прежний слот.
8. Защита от конфликтов. Обеспечить ограничение на количество записей пользователя и исключить ситуацию, когда два пользователя одновременно бронируют один слот. Для этого необходимы транзакции и ограничения целостности в базе данных.
9. Уведомление. После успешного фиксирования изменений создать событие для отправки уведомления пользователю по предусмотренному каналу. Сбой доставки уведомления не должен отменять уже выполненный перенос.
10. Формирование ответа. Вернуть 200 OK с новой датой, статусом и временем изменения.

11. Обработка повторов. Если запрос с тем же Idempotency-Key уже успешно обработан, вернуть сохранённый результат без повторного переноса. При повторном использовании ключа с другим содержимым вернуть 409 Conflict.

5. Сводная таблица всех методов API

№	HTTP-метод	Endpoint	Назначение	Успешный код
1	POST	`/auth/login`	Вход в систему	`200 OK`
2	GET	`/me`	Получение профиля	`200 OK`
3	POST	`/auth/logout`	Выход из системы	`204 No Content`
4	GET	`/tests`	Список тестов	`200 OK`
5	GET	`/tests/{testId}`	Информация о тесте	`200 OK`
6	GET	`/tests/{testId}/questions`	Вопросы теста	`200 OK`
7	POST	`/test-attempts`	Начало прохождения	`201 Created`
8	PUT	`/test-attempts/{attemptId}/answers`	Сохранение ответов	`200 OK`
9	POST	`/test-attempts/{attemptId}/submit`	Завершение теста	`200 OK`
10	GET	`/test-attempts/{attemptId}`	Получение результата	`200 OK`
11	GET	`/me/interview`	Текущая запись на собеседование	`200 OK`
12	GET	`/interview-slots`	Доступные слоты	`200 OK`
13	PUT	`/me/interview/reschedule`	Перенос собеседования	`200 OK`

6. Общие коды ответов API

6.1. Коды успешных ответов

HTTP-код	Название	Когда используется
`200 OK`	Успешно	Запрос выполнен; возвращены данные операции
`201 Created`	Создано	Создана новая попытка прохождения теста

204 No Content	Нет содержимого	Успешный выход из системы без тела ответа

6.2. Коды ошибок клиента

HTTP-код	Название	Причина	Пример использования
400 Bad Request	Некорректный запрос	Некорректный JSON, синтаксическая ошибка или некорректный формат запроса	Тело запроса невозможно разобрать
401 Unauthorized	Не авторизован	Токен отсутствует, недействителен или истёк; неверные учётные данные	Вход или доступ к защищённому ресурсу
403 Forbidden	Доступ запрещён	Пользователь авторизован, но не имеет необходимых прав	Доступ к закрытому тесту
404 Not Found	Не найдено	Ресурс отсутствует или недоступен в текущем контексте	Тест, попытка, запись или слот не найдены
409 Conflict	Конфликт	Операция противоречит текущему состоянию ресурса	Занятый слот, превышение числа попыток, завершённая попытка
415 Unsupported Media Type	Неподдерживаемый тип	Передан неподдерживаемый формат тела запроса	Вместо JSON передан XML
422 Unprocessable Content	Ошибка проверки данных	Запрос синтаксически корректен, но нарушает правила валидации	Некорректные ответы или период поиска слотов
429 Too Many Requests	Слишком много запросов	Превышен лимит частоты запросов	Частые попытки входа

6.3. Коды ошибок сервера

HTTP-код	Название	Причина	Пример использования
500 Internal Server Error	Внутренняя ошибка	Необработанная ошибка приложения	Непредвиденная ошибка при обработке запроса
502 Bad Gateway	Ошибка шлюза	Ошибка взаимодействия с внешним сервисом	Недоступен промежуточный сервис, через который выполняется запрос
503 Service	Сервис	Временная недоступность	База данных или критическая

Unavailable	недоступен	сервиса или перегрузка	зависимость временно недоступна
504 Gateway Timeout	Тайм-аут шлюза	Превышено время ожидания ответа от зависимого сервиса	Внешняя система не ответила вовремя

7. Описание метода /auth/login в виде спецификации OpenAPI 3

```
openapi: 3.0.3
info:
  title: RESTful API веб-сервиса «Волонтёрский центр»
  version: 1.0.0
  description: Спецификация метода авторизации пользователя
servers:
  - url: https://api.volunteer-center.ru/api/v1
    description: Базовый URL API
tags:
  - name: Авторизация
    description: Методы входа и выхода из системы
paths:
  /auth/login:
    post:
      tags:
        - Авторизация
      summary: Вход в систему
      description: >-
        Проверка учётных данных пользователя и выдача токена доступа.
        Метод не требует предварительной авторизации.
      operationId: login
      requestBody:
        required: true
```

```
        description: Учётные данные пользователя
        content:
          application/json:
            schema:
              type: object
              required:
                - login
                - password
              properties:
                login:
                  type: string
                  minLength: 3
                  maxLength: 254
                  description: Логин или адрес электронной почты без
пробелов по краям
                  example: petr.petrov@example.ru
                password:
                  type: string
                  minLength: 8
                  maxLength: 128
                  description: Пароль пользователя
                  example: qwerty123
            example:
              login: petr.petrov@example.ru
              password: qwerty123
      responses:
        '200':
          description: Успешная авторизация
          content:
            application/json:
              schema:
                type: object
```

```
required:
  - accessToken
  - tokenType
  - expiresIn
  - user
properties:
  accessToken:
    type: string
    description: Токен доступа
    example: eyJhbGciOiJIUzI1N
  tokenType:
    type: string
    enum:
      - Bearer
    description: Тип токена
    example: Bearer
  expiresIn:
    type: integer
    minimum: 1
    description: Время действия токена в секундах
    example: 3600
  user:
    type: object
    required:
      - id
      - fullName
      - email
      - role
    properties:
      id:
        type: string
```

```
                format: uuid
                description: Идентификатор пользователя
                example: a1b2c3d4-e5f6-4789-8123-456789abcd01
              fullName:
                type: string
                minLength: 1
                maxLength: 200
                description: Отображаемое имя пользователя
                example: Петр Петров
              email:
                type: string
                format: email
                description: Электронная почта
                example: petr.petrov@example.ru
              role:
                type: string
                description: Роль пользователя
                example: volunteer
      example:
        accessToken: eyJhbGciOiJIUzI1N
        tokenType: Bearer
        expiresIn: 3600
        user:
          id: a1b2c3d4-e5f6-4789-8123-456789abcd01
          fullName: Петр Петров
          email: petr.petrov@example.ru
          role: volunteer
'401':
  description: Неверный логин или пароль
  content:
    application/json:
```

```
            schema:
              $ref: '#/components/schemas/ErrorResponse'
            example:
              error:
                code: INVALID_CREDENTIALS
                message: Неверный логин или пароль
        '422':
          description: Некорректные входные данные
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ErrorResponse'
              example:
                error:
                  code: VALIDATION_ERROR
                  message: Ошибка проверки входных данных
                  details:
                    - field: login
                      message: Поле обязательно для заполнения
        '429':
          description: Превышено количество попыток входа
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ErrorResponse'
              example:
                error:
                  code: TOO_MANY_REQUESTS
                  message: Слишком много попыток входа. Повторите попытку
позже
components:
  schemas:
```

```
ErrorResponse:
  type: object
  required:
    - error
  properties:
    error:
      type: object
      required:
        - code
        - message
      properties:
        code:
          type: string
          description: Код ошибки
          example: VALIDATION_ERROR
        message:
          type: string
          description: Описание ошибки
          example: Ошибка проверки входных данных
        details:
          type: array
          description: Дополнительные сведения об ошибках полей
          items:
            type: object
            required:
              - field
              - message
            properties:
              field:
                type: string
                description: Название поля с ошибкой
```

```
    example: login
  message:
    type: string
    description: Описание ошибки поля
    example: Поле обязательно для заполнения
```